Introduction to Pipelining



Pipelining & Parallelism



Most high-performance computers exhibit a great deal of concurrency.

However, it is not desirable to call every modern computer a parallel computer.

Pipelining and parallelism are 2 methods used to achieve concurrency.

Pipelining increases concurrency by dividing a computation into a number of steps.

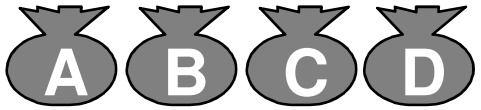
Parallelism is the use of multiple resources to increase concurrency.

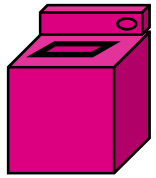
Pipelining is Natural!

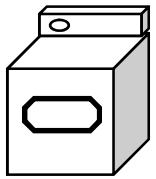


- Laundry Example
- Ann, Brian, Cathy, Dave each have one load of clothes to wash, dry, and fold
- Washer takes 30 minutes
- Dryer takes 30 minutes
- “Folder” takes 30 minutes
- “Stasher” takes 30 minutes to put clothes into drawers

把折疊好的衣服放進密封袋的人員



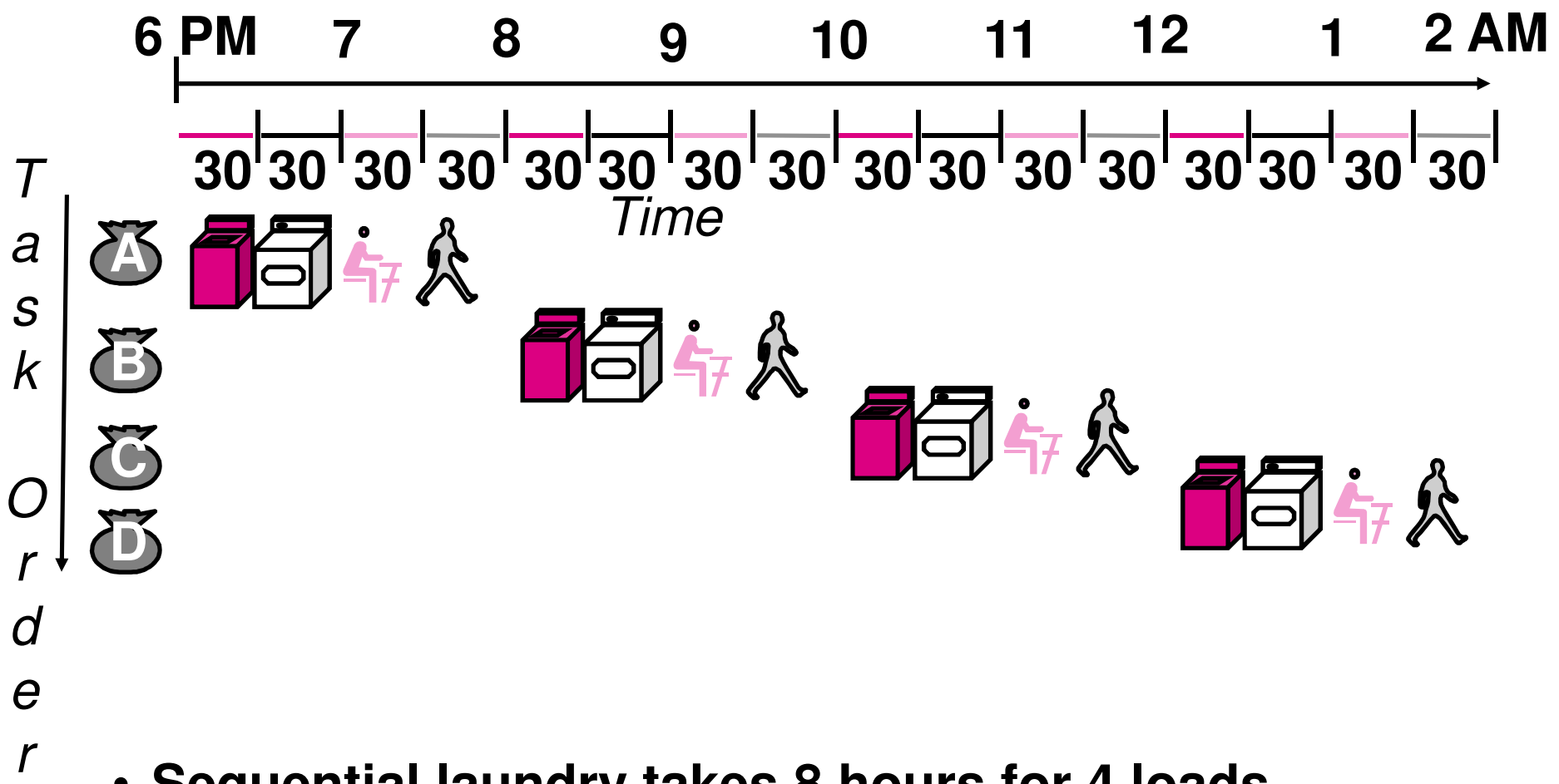






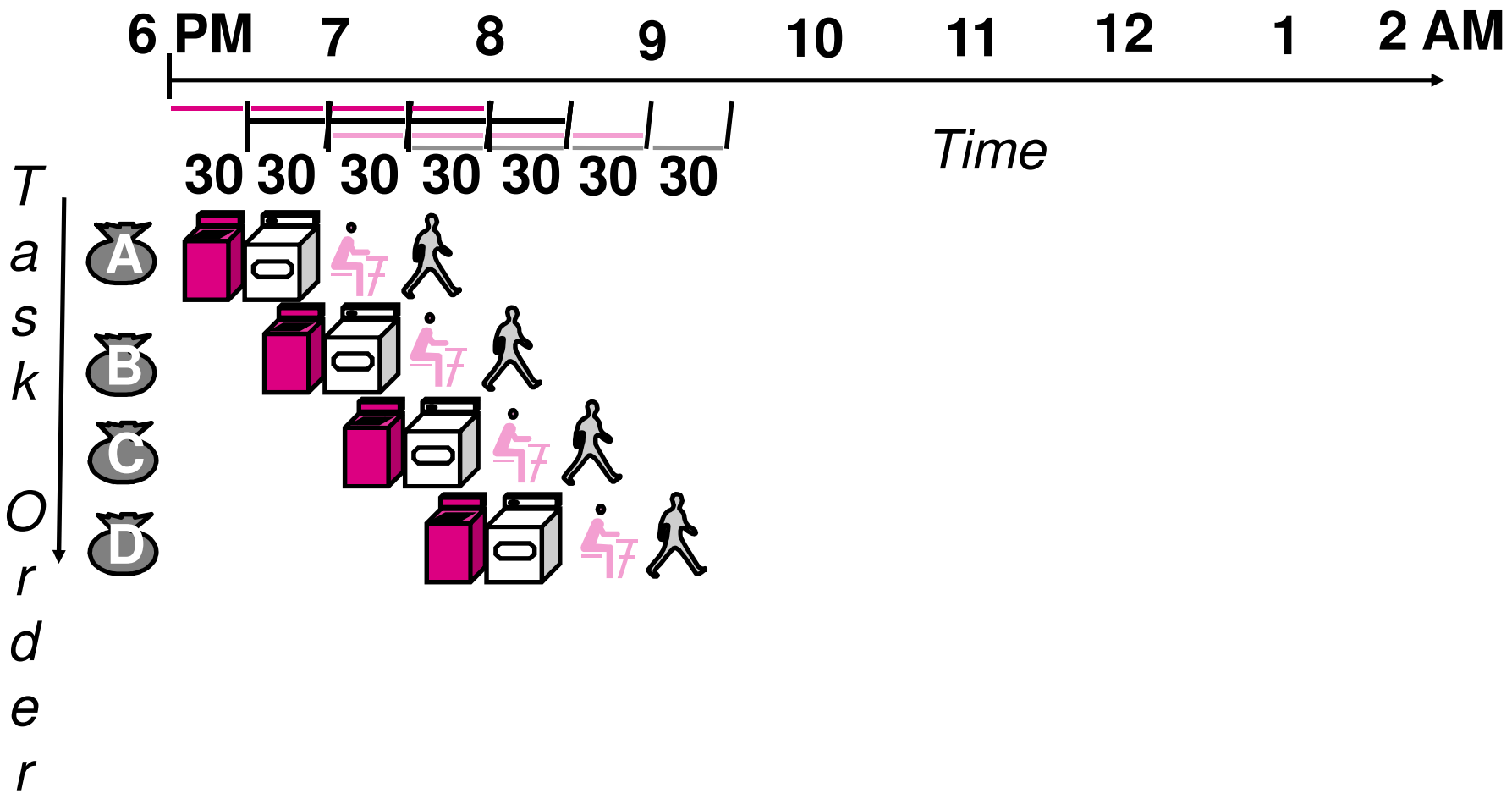


Sequential Laundry



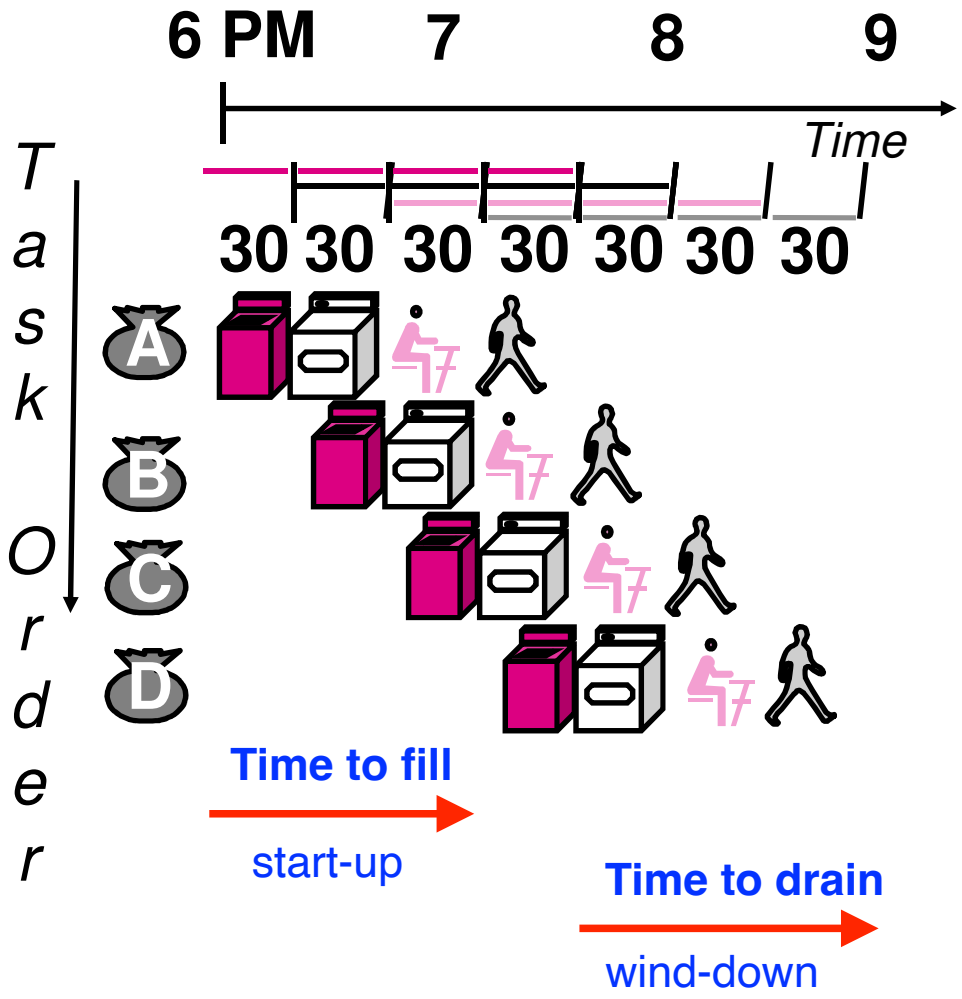
- Sequential laundry takes 8 hours for 4 loads
- If they learned pipelining, how long would laundry take?

Pipelined Laundry: Start work ASAP



- **Pipelined laundry takes 3.5 hours for 4 loads!**

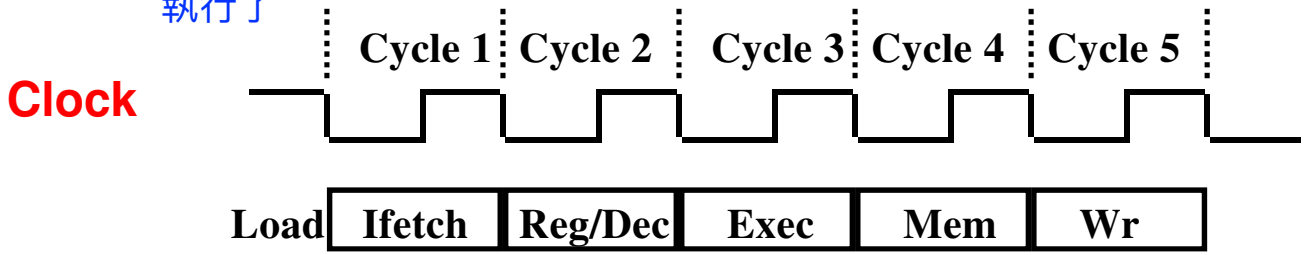
Pipelining Lessons



- Pipelining doesn't help latency of single task, it helps throughput of entire workload
- Multiple tasks operating simultaneously using different resources
- Potential speedup = Number of pipe stages
- Pipeline rate limited by slowest pipeline stage
- Unbalanced lengths of pipe stages reduces speedup
- Time to "fill" pipeline and time to "drain" it reduces speedup
- Stall for Dependences
  熄火

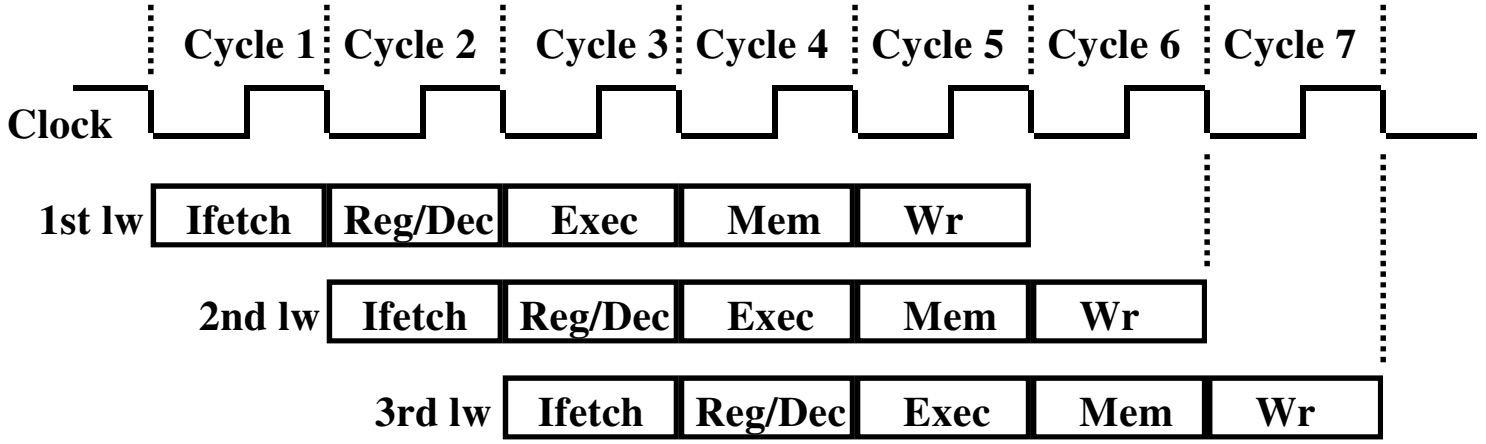
5 Execution Stages of lw Instruction

上一章節T2的動作ALUOut = PC + (sign-extend(IR[15-0]) << 2)
由beq的指令在執行階段再執行此一動作，其餘不相干指令就不執行了



- **Ifetch: Instruction Fetch** — lw $s1, 100($s0)
  - **–Fetch instruction from Instruction Memory**
- **Reg/Dec: Registers Fetch and Instruction Decode** — lw $s0
- **Exec: Calculation of memory address** — 100 + Reg[$s0]
- **Mem: Read data from Data Memory** — MDR = Mem[100 + Reg[$s0]]
- **Wr: Write data back to register file** — Reg[$s0] = MDR

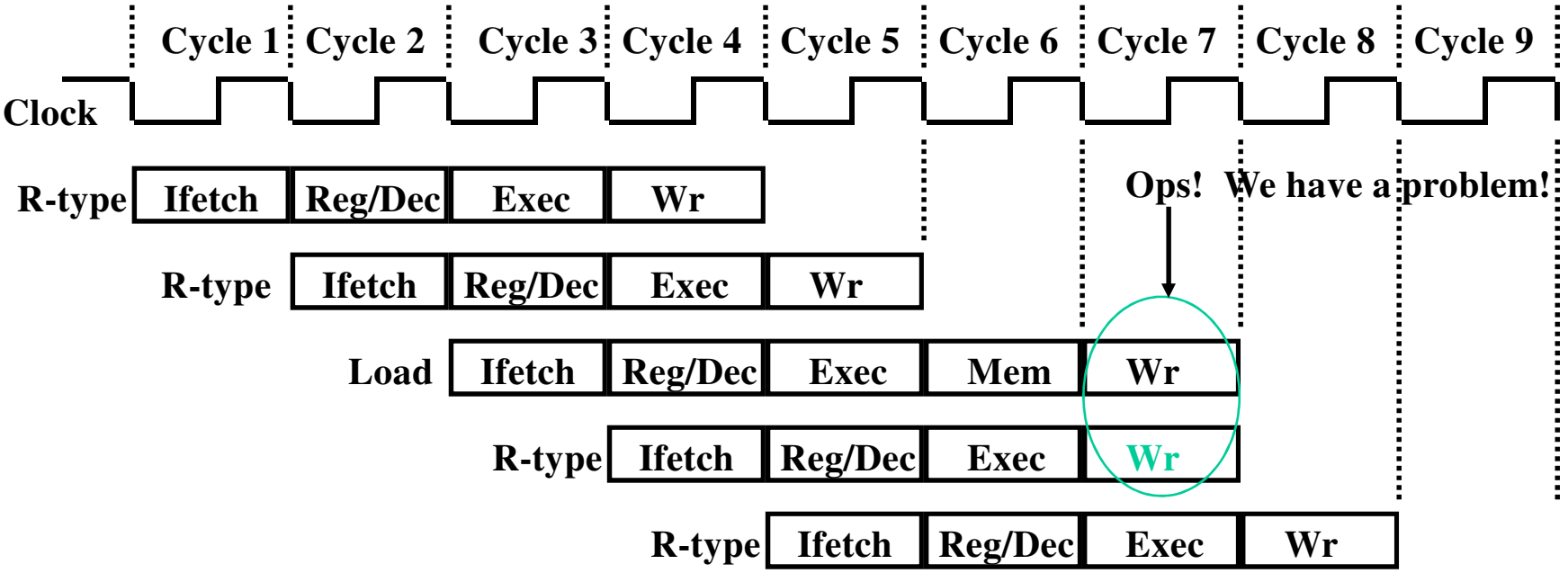
Load Instruction in pipelined Implementation



採用Harvard架構

- 5 independent functional units in pipeline datapath
  - –Instruction Memory for the Ifetch stage
  - –Register File's Read ports (bus A and busB) for the Reg/Dec stage
  - –ALU for the Exec stage
  - –Data Memory for the Mem stage
  - –Register File's Write port (bus W) for the Wr stage

R-type Instruction in Pipelined Implementation

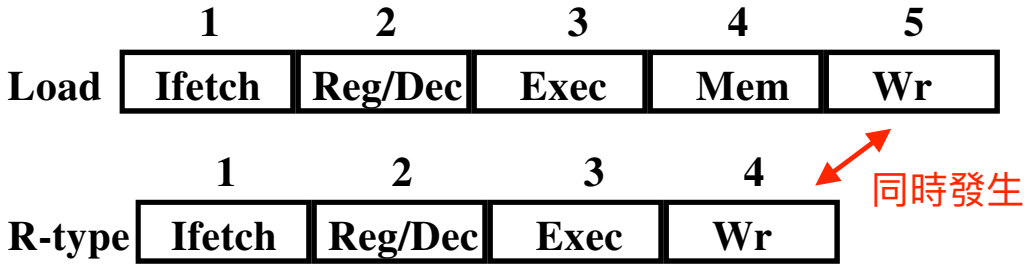


- **Pipeline Conflict**(Structural Hazard) 結構性危險
  - –Two instructions try to write to register file at the same time!
  - –Only one write port

Pipeline Hazard

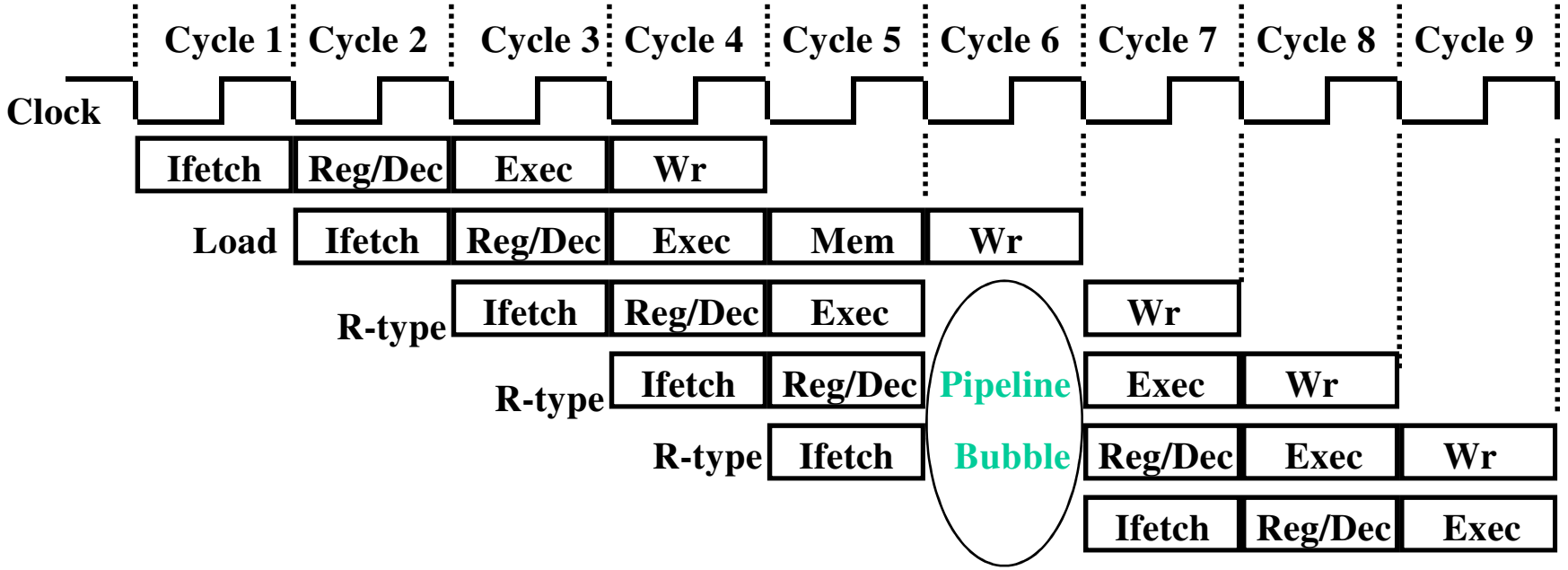


- Each functional unit can only be used once per instruction
- Each functional unit must be used at the same stage for all instructions:
  - Load Instruction uses Register File's Write Port during its 5th stage
  - R-type Instruction uses Register File's Write Port during its 4th stage



pipeline stall

Solution 1 for Pipeline Hazard: "Bubble" Insertion

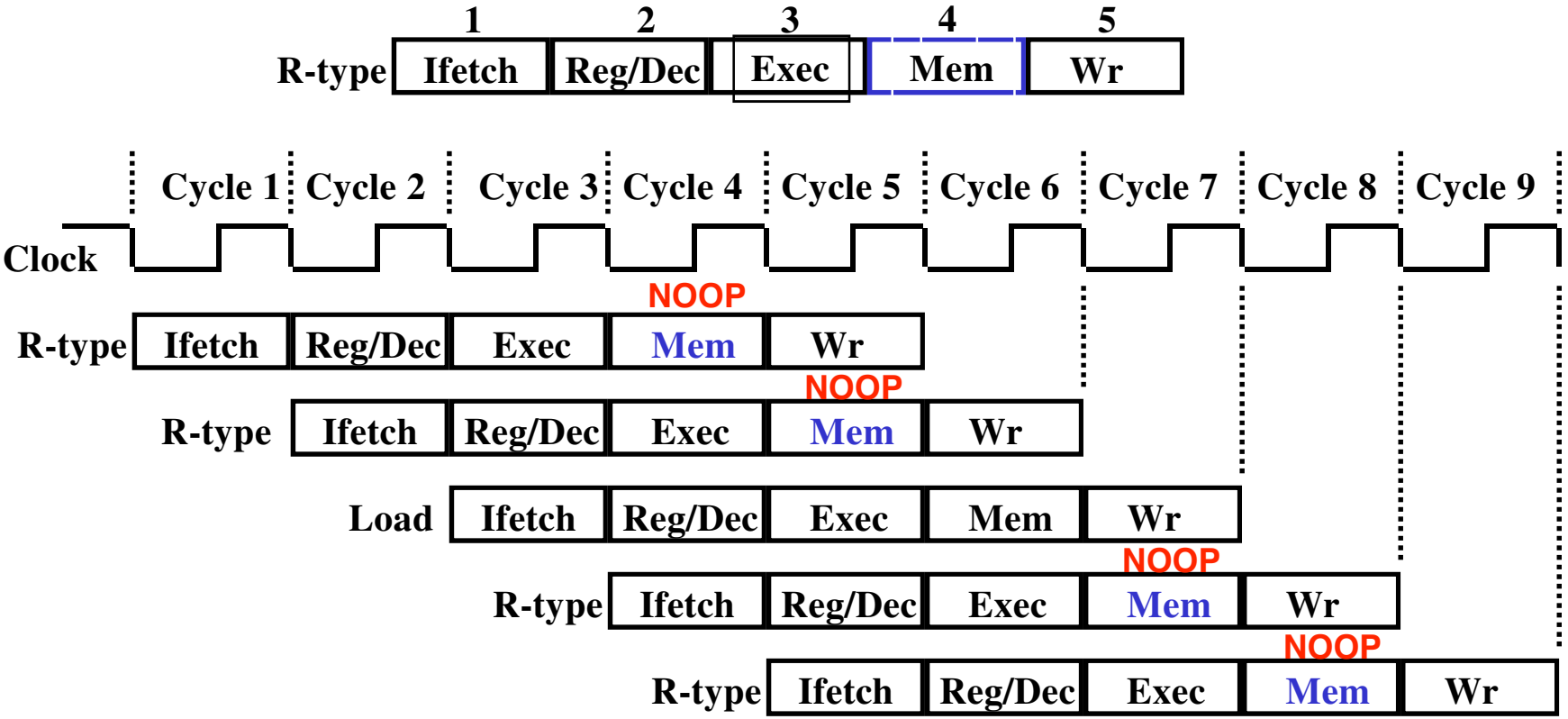


- Insert a "bubble" into the pipeline to prevent 2 writes at the same cycle
  - –Control logic can be complex.
  - –Lose instruction fetch
- No instruction is started in Cycle 6!

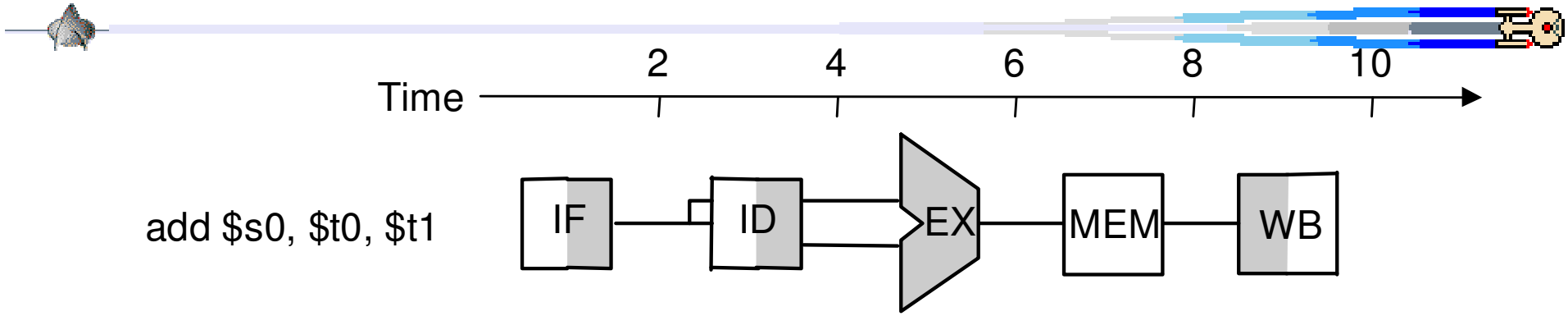
Solution 2 for Pipeline Hazard: Delay R-type's Write

- **Delay R-type's register write by one cycle:**
  - **R-type instructions also use Reg File's write port at Stage 5**
  - **Mem stage is a NOOP stage: nothing is being done.**

NO OPeration



Instruction Pipeline Representation



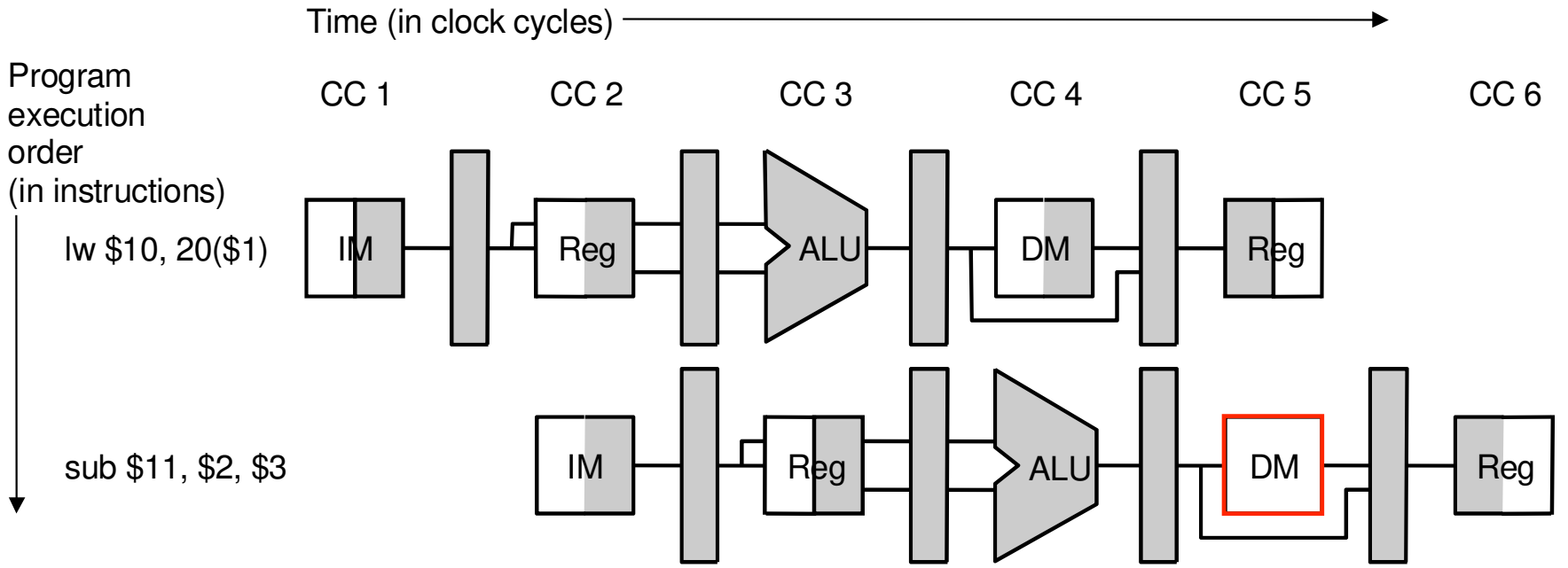
Pipeline stages

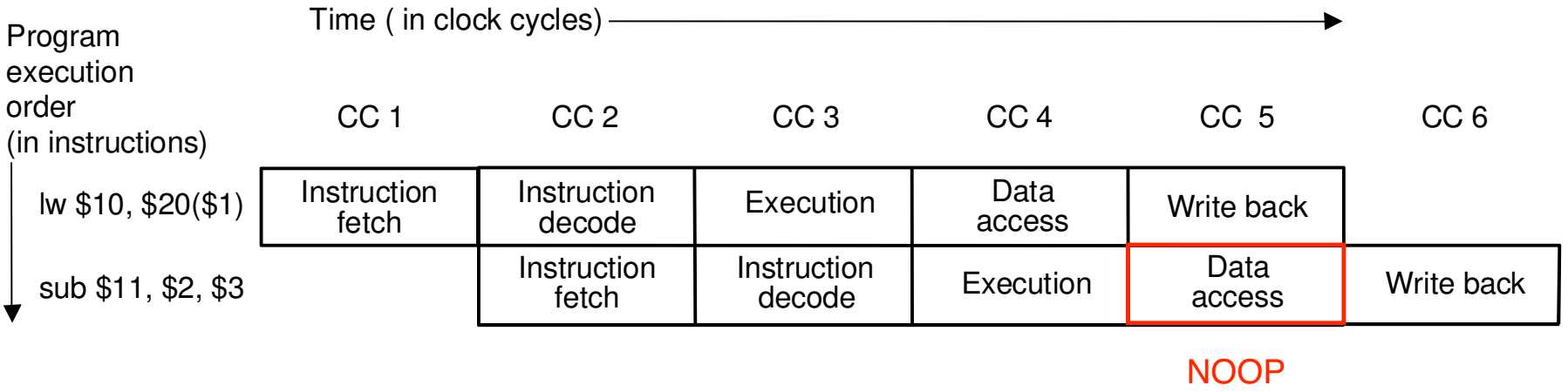
- IF instruction fetch (read)
- ID instruction decode and register read (read)
- EX execute alu operation
- MEM data memory (read or write)
- WB Write back to register

Resources

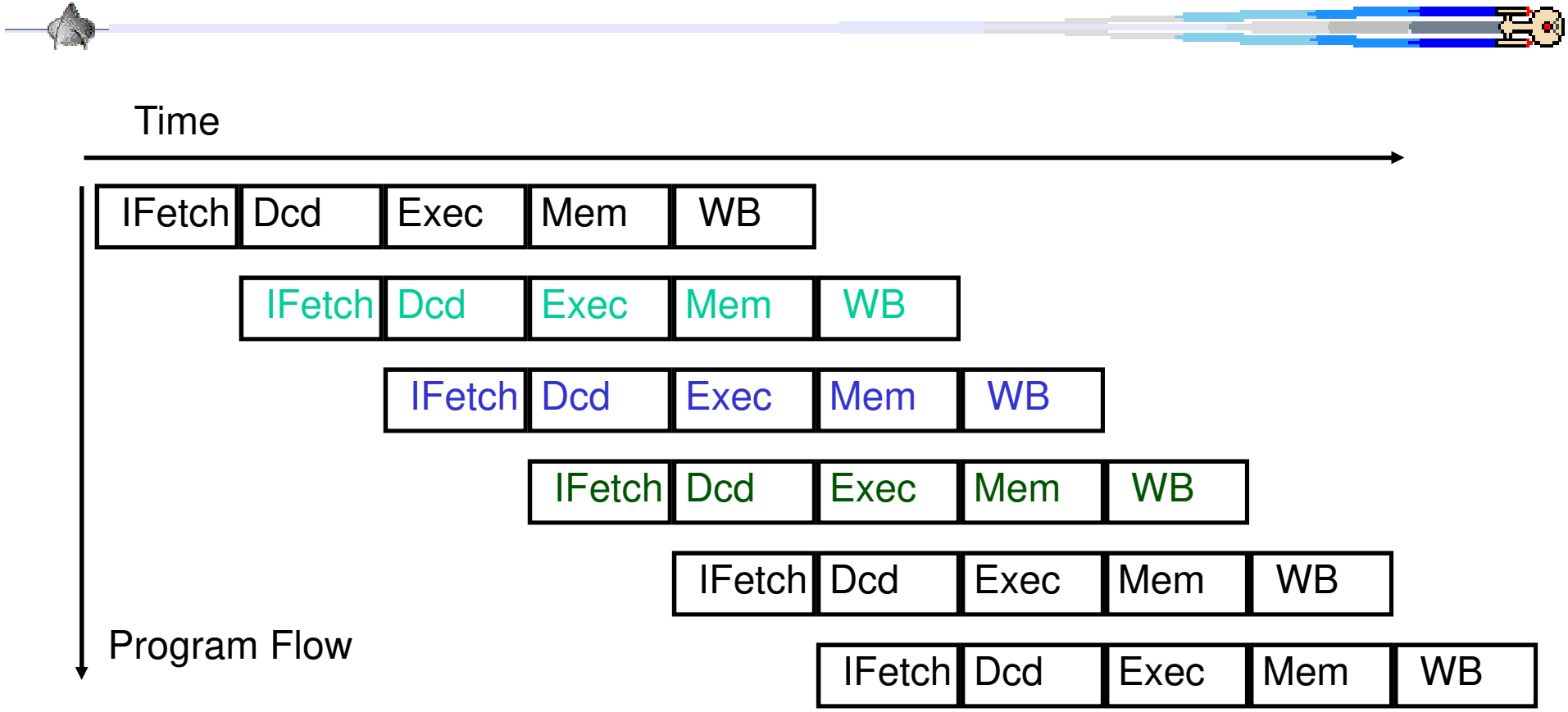
- Mem — IM & DM instr. & data memory
- RegRead1 — register read port #1
- RegRead2 — register read port #2
- RegWrite — register write
- ALU — alu operation

Multiple-Instruction Pipeline Diagram

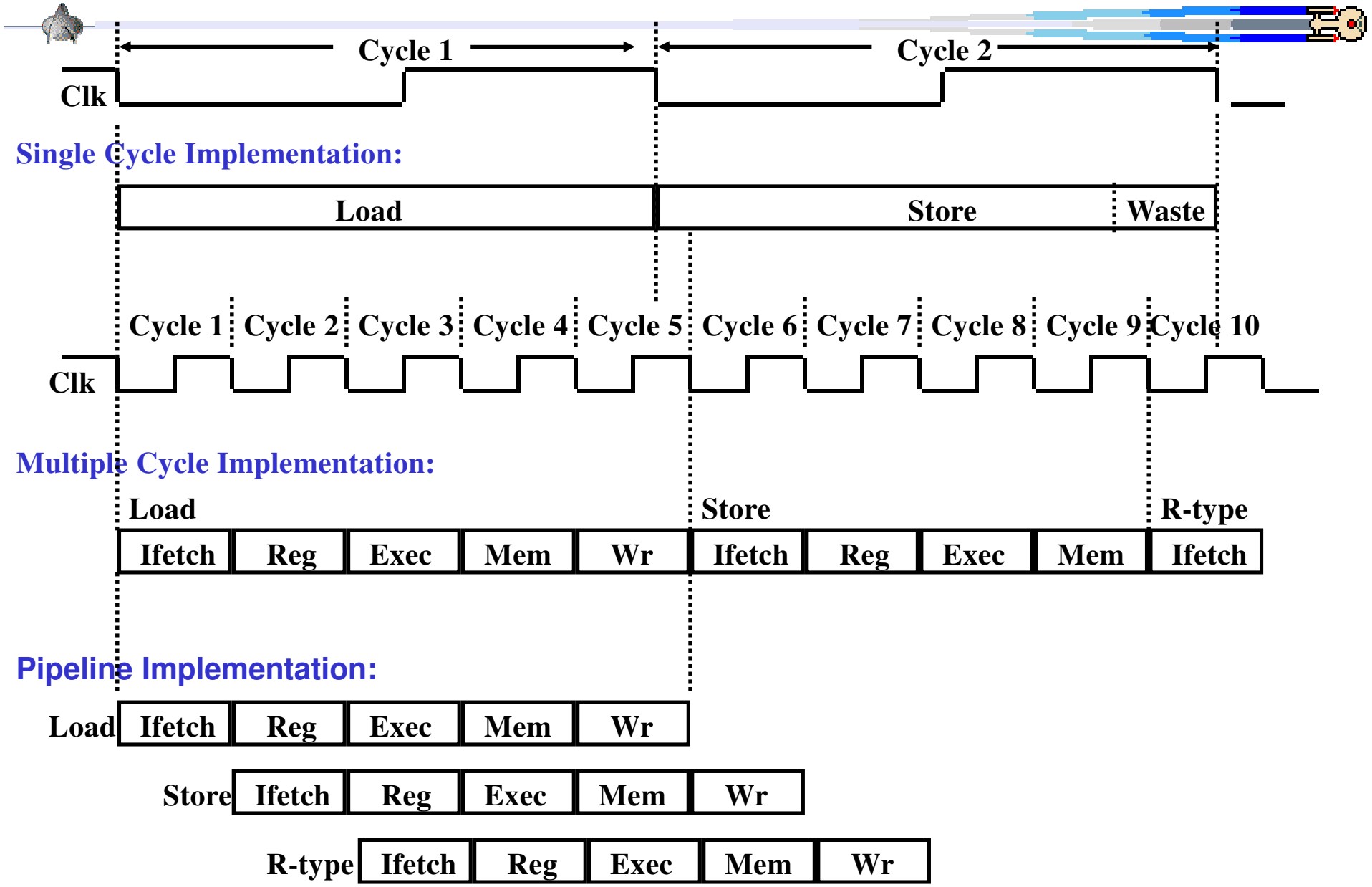




Pipelined Execution Representation



Single-Cycle, Multiple-Cycle, & Pipeline Execution



Reason for Pipeline Execution: Timing



- Suppose we execute 100 instructions
- Single Cycle Machine
  - 45 ns/cycle x 1 CPI x 100 inst = 4500 ns
- Multicycle Machine
  - 10 ns/cycle x 4.6 CPI (due to inst mix) x 100 inst = 4600 ns

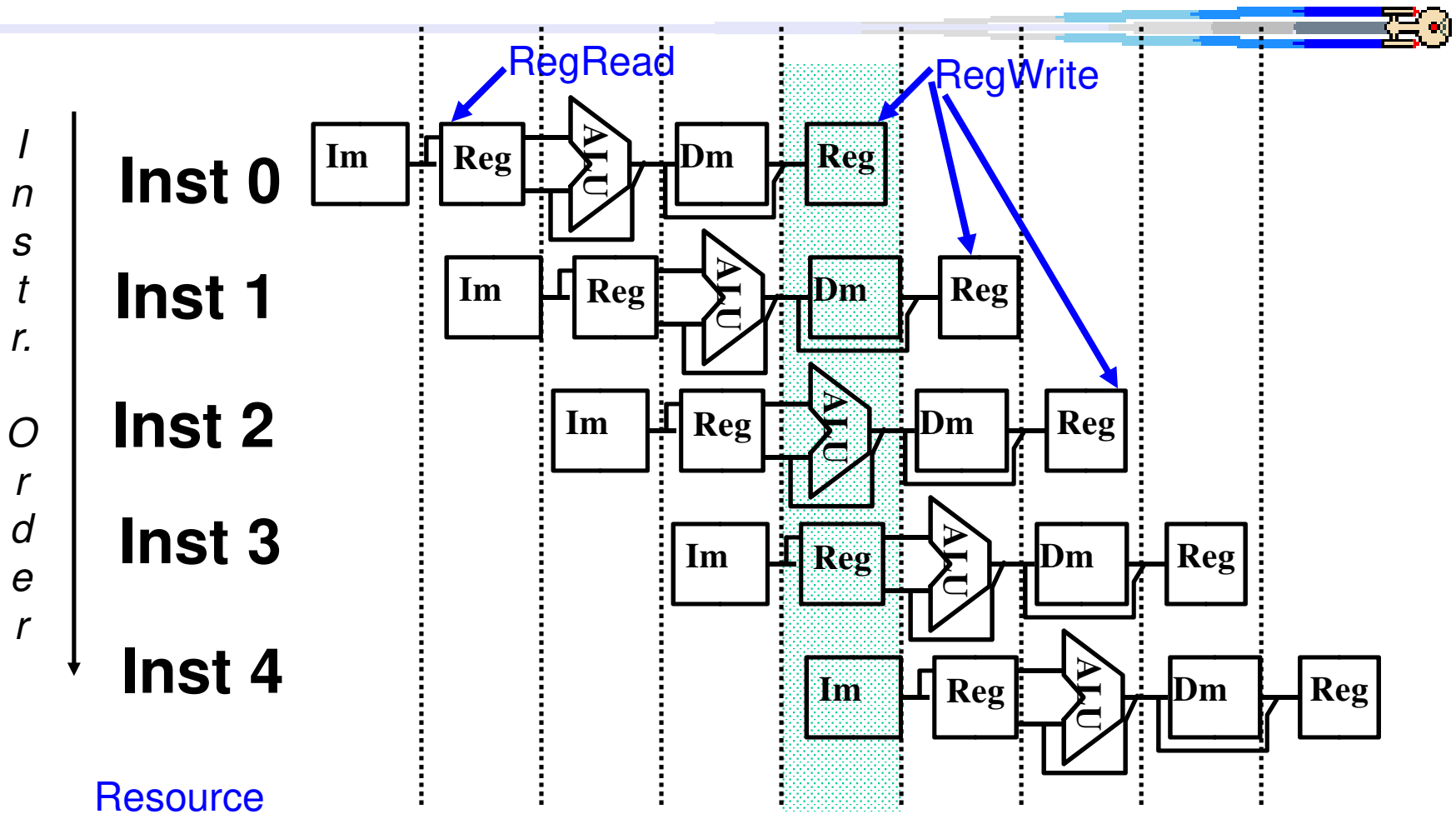
4.6是100個指令中假設的平均值CPI

- Ideal pipelined machine
  - 10 ns/cycle x (1 CPI x 100 inst + 4 cycle drain) = 1040 ns

一開始的前4個cycle只有執行，沒有完成任何指令。
從第5個cycle開始，每一個cycle都可以完成一個指令。

儘可能不要讓元件閒置

Reason for Pipeline Execution: Resources Usage



Resource									
MemInst	busy	busy	busy	busy	busy	idle	idle	idle	idle
MemData	idle	idle	idle	busy	busy	busy	busy	busy	idle
RegRead	idle	busy	busy	busy	busy	busy	idle	idle	idle
RegWrite	idle	idle	idle	idle	busy	busy	busy	busy	busy
ALU	idle	idle	busy	busy	busy	busy	busy	idle	idle

Pipeline Hazards



- Structural Hazards: attempt to use the same resource two different ways at the same time
  - E.g., combined washer/dryer would be a structural hazard or folder busy doing something else (watching TV)

- Data Hazards: attempt to use item before it is ready
  - E.g., one sock of pair in dryer and one in washer; can't fold until get sock from washer through dryer
  - instruction depends on result of prior instruction still in the pipeline

- Control Hazards: attempt to make a decision before condition is evaulated
  - E.g., washing football uniforms and need to get proper detergent level; need to see after dryer before next load in
  - branch instructions

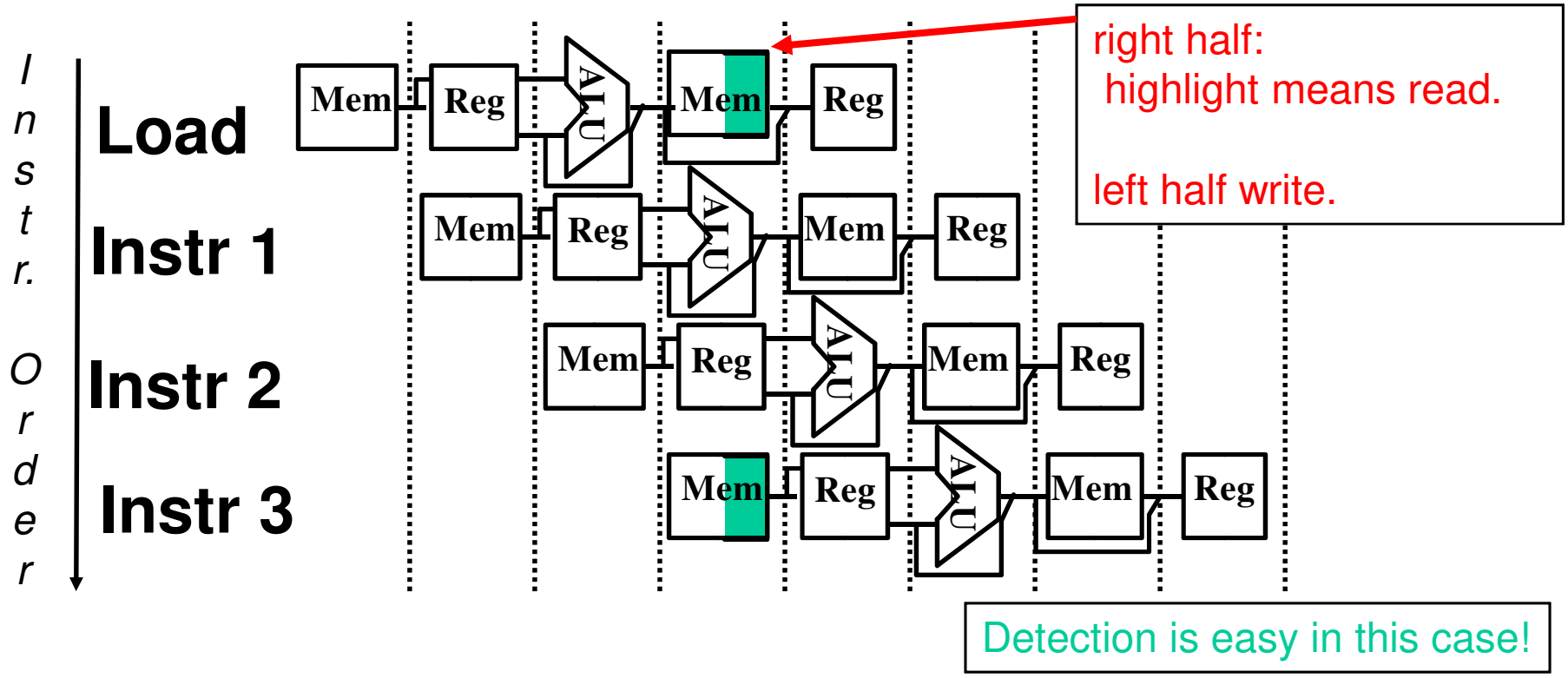
足球衣服 洗滌劑 洗滌衣服之前要先閱讀洗滌注意事項

- **Simple solution for hazards by waiting**
  - pipeline control must detect the hazard
  - take action (or delay action) to resolve hazards

Structural Hazard

Harvard

Previous example: Separate InstMem and DataMem



	Resource								
	Mem(Inst & Data)	busy	busy	busy	busy	busy	idle	idle	idle
von Neuman	RegRead	idle	busy	busy	busy	busy	busy	idle	idle
	RegWrite	idle	idle	idle	idle	busy	busy	busy	busy
	ALU	idle	idle	busy	busy	busy	busy	busy	idle

Solutions for Structural Hazard



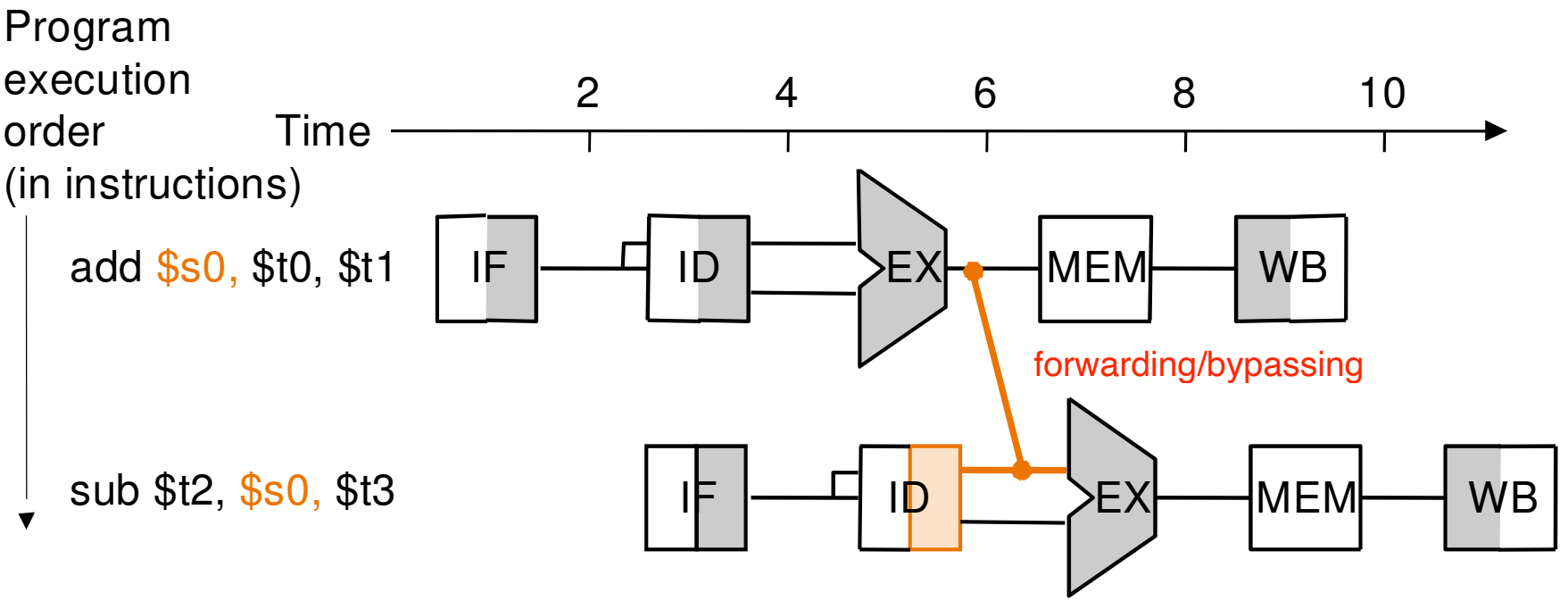
structural hazards: attempt to use the same resource two different ways at the same time

By change the architecture from a Harvard (separate instruction and data memory) to a von Neuman memory, we actually created a structural hazard!

Structural hazards can be avoid by changing

- hardware: design of the architecture (splitting resources)
- software: re-order the instruction sequence
- software: delay

Data Dependency & Data Hazard



Adding extra hardware to retrieve the missing item early from the internal resources is called forwarding or bypassing.

Forwarding works very well. It cannot prevent all pipeline stalls, however.

Figure 6.8

Data Dependency on r1



```
add r1 ,r2,r3

sub r4, r1 ,r3

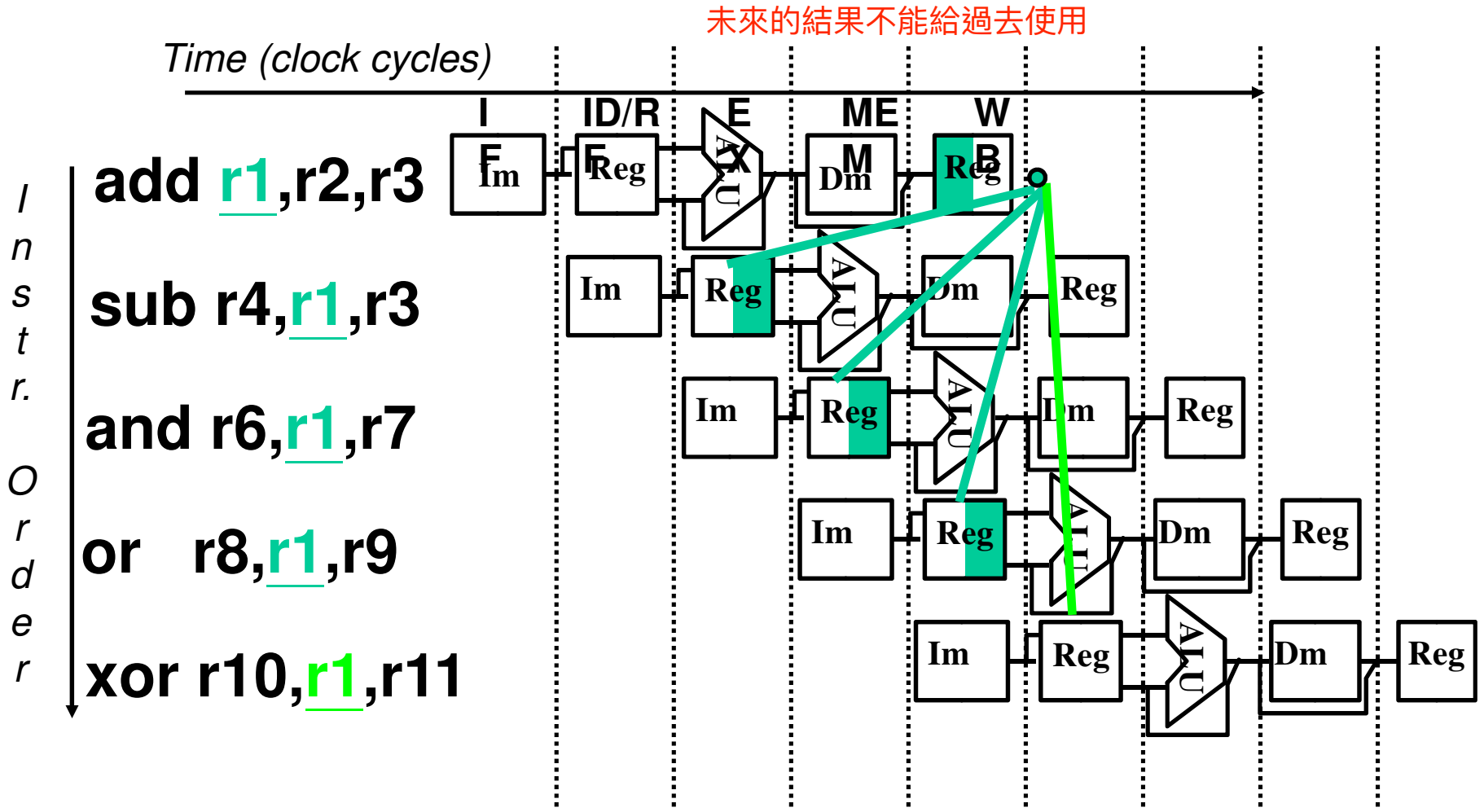
and r6, r1 ,r7

or   r8, r1 ,r9

xor r10, r1 ,r11
```

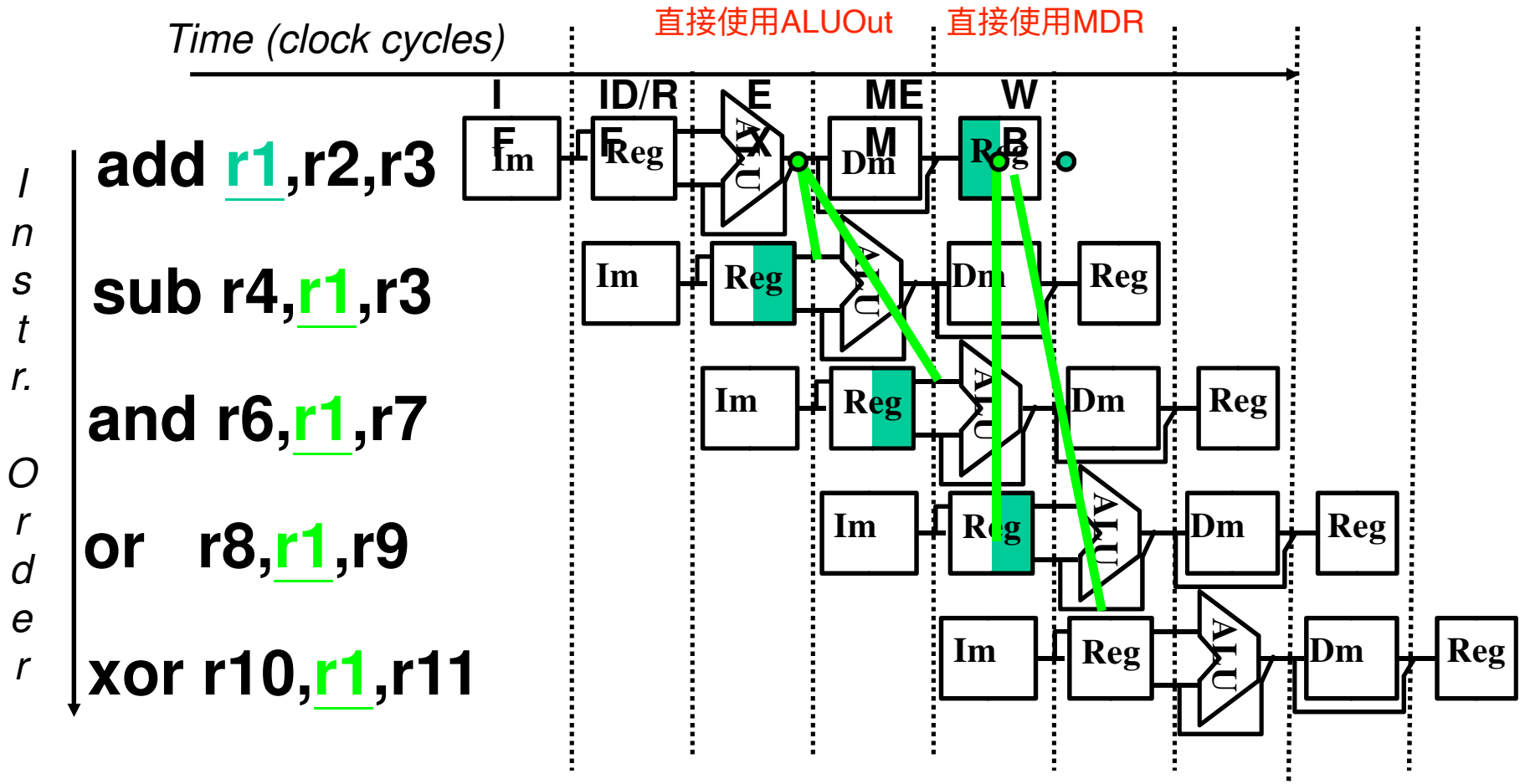
Data Hazard on Data r1

- **Dependencies backwards in time are hazards**



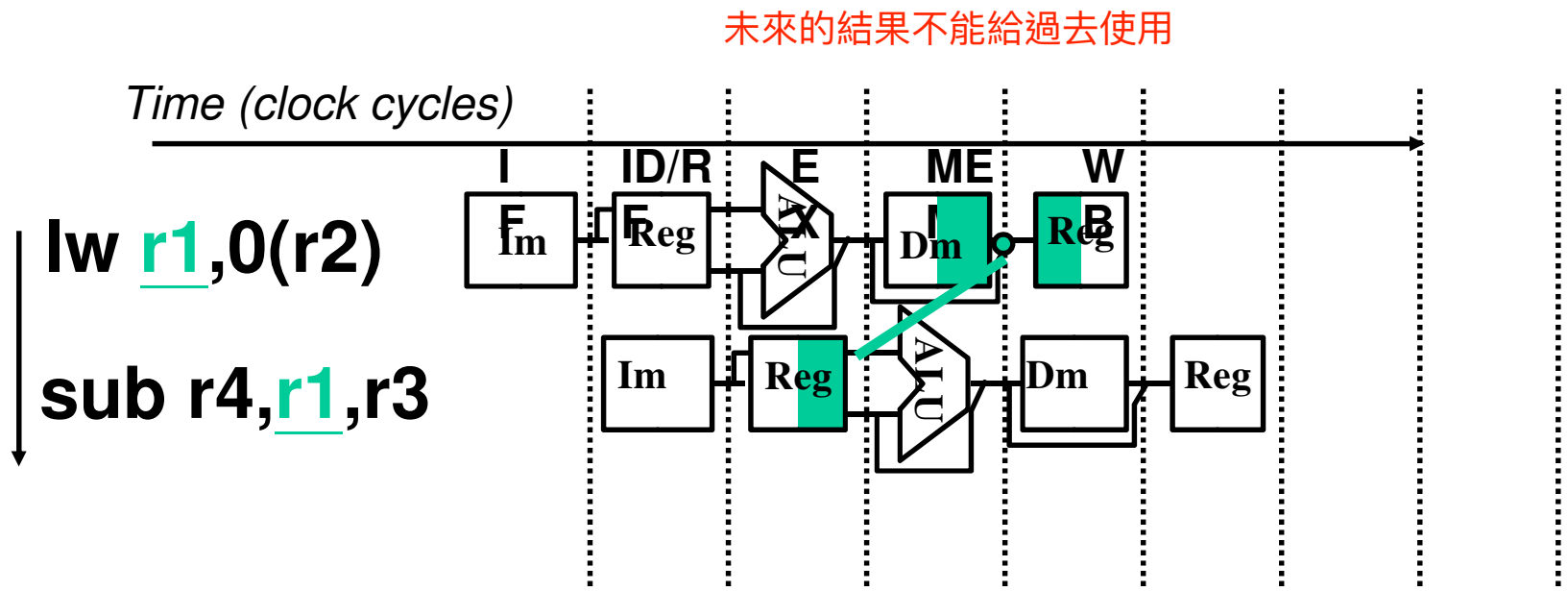
Forward Solution for Data Hazard

- "Forward" result from one stage to another

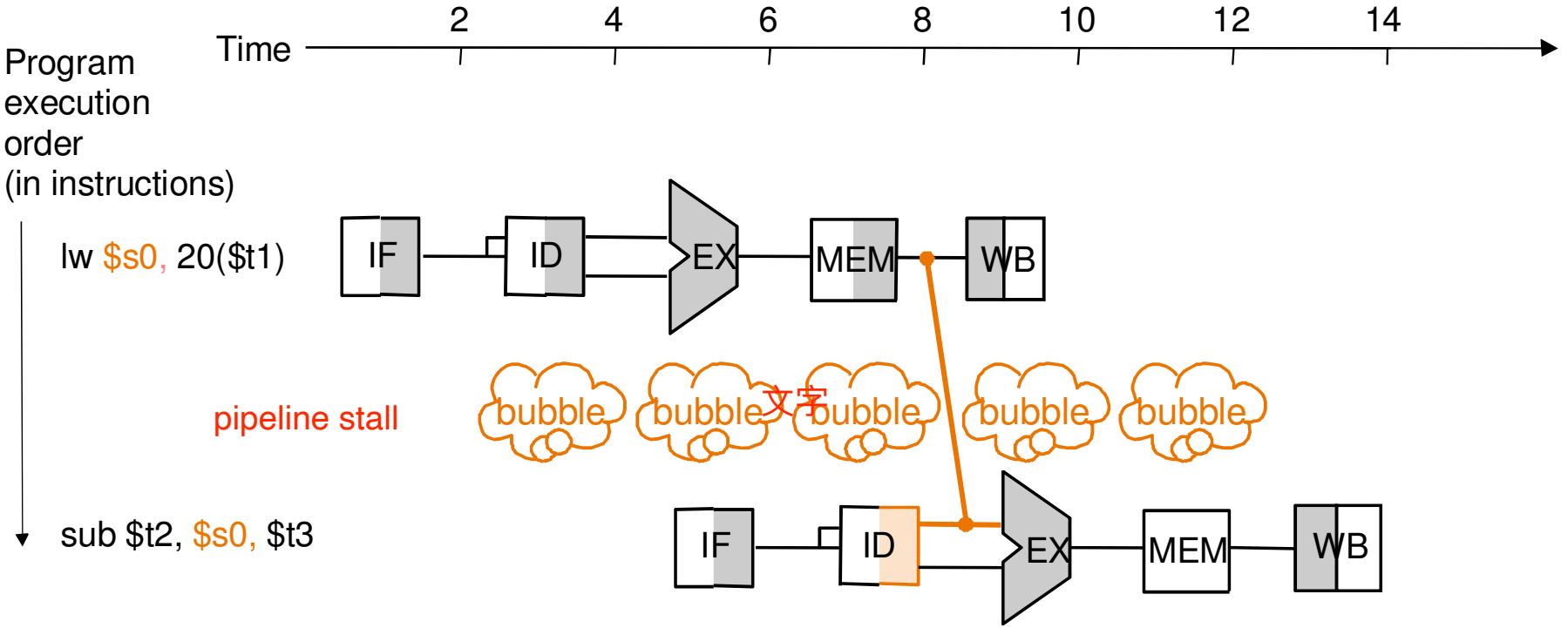


Forwarding for Load Instruction

- Dependencies backwards in time are hazards
- Delay/stall instruction dependent on loads



Stall & Forwarding for Load Data Hazard



Instruction Reordering for Data Hazard



```
lw      $t0, 0($t1)        $t0=Memory[0+$t1]
lw      $t2, 4($t1)        $t2=Memory[4+$t1]
sw      $t2, 0($t1)        Memory[0+$t1]=$t2
sw      $t0, 4($t1)        Memory[4+$t1]=$t0
```

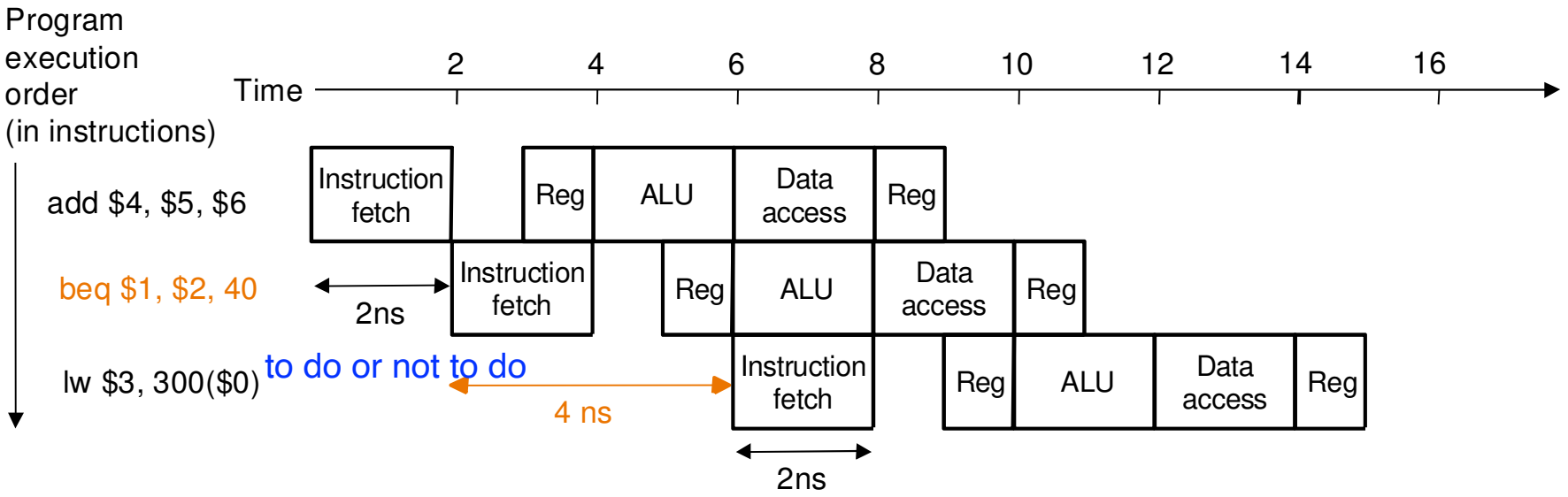


```
lw      $t2, 4($t1)
lw      $t0, 0($y1)
sw      $t2, 0($t1)
sw      $t0, 4($t1)
```

Figure 6.9

Control Hazard & Stall on Branch Instruction



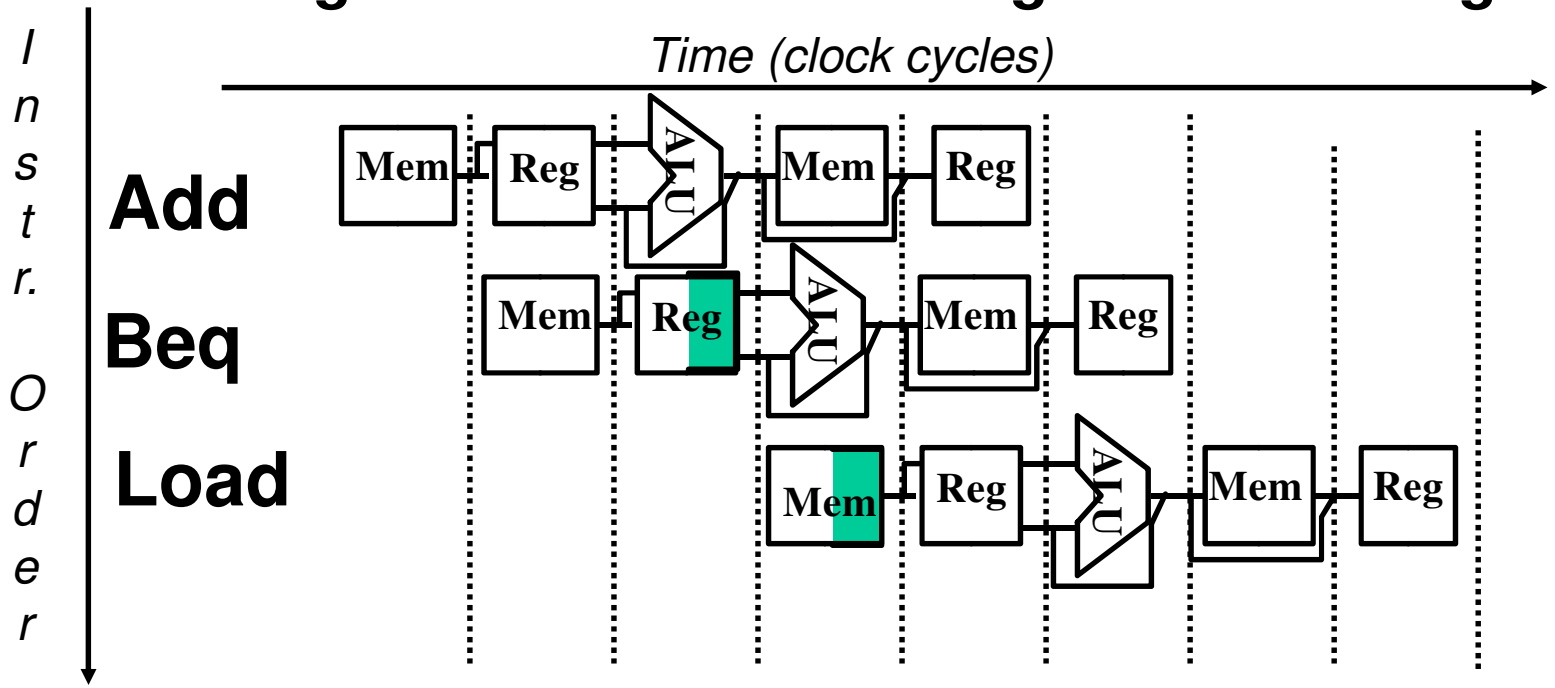


put in enough extra hardware so that we can test registers, calculate the branch address, and update the PC during the second stage of the pipeline

Figure 6.4

Stall Solution for Control Hazard

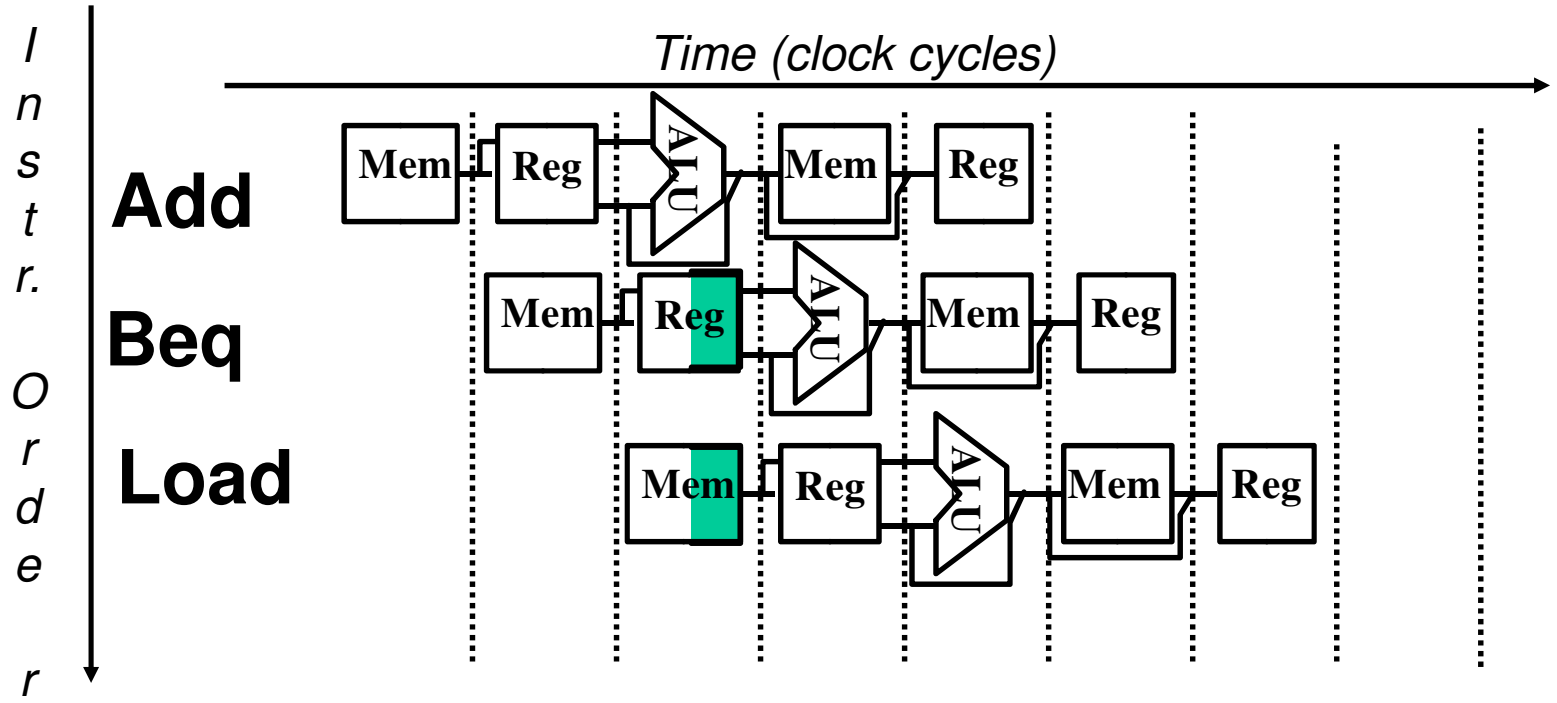
- Stall: wait until decision is clear
  - –Its possible to move up decision to 2nd stage by adding hardware to check registers as being read



- Impact: 2 clock cycles per branch instruction => slow

Prediction Solution for Control Hazard

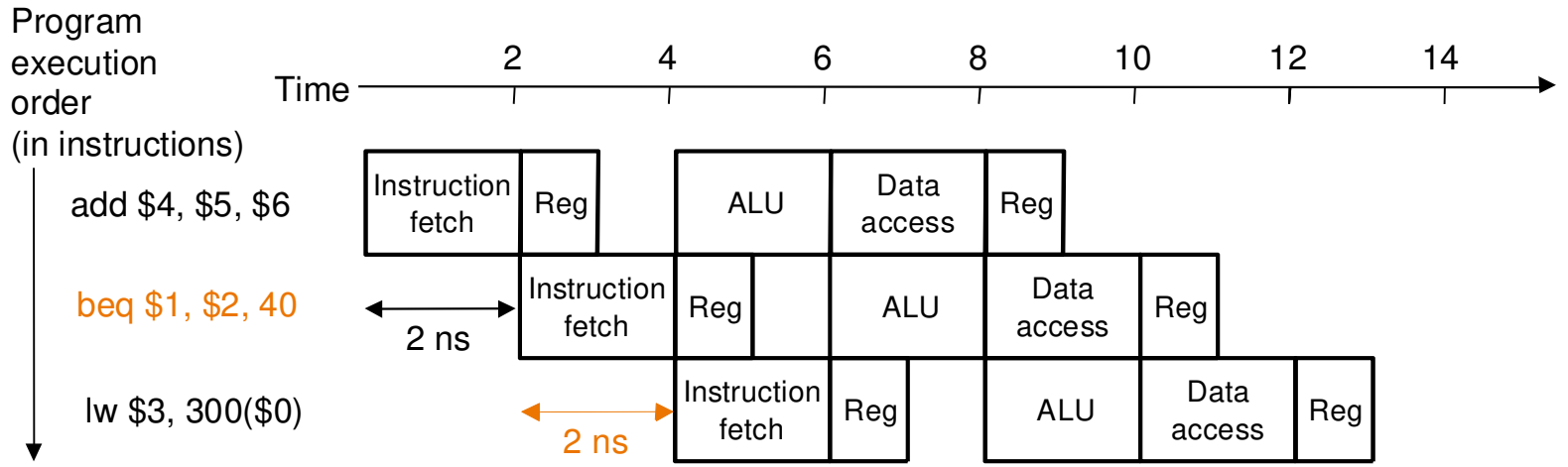
- **Predict: guess one direction then back up if wrong**
  - **–Predict not taken**

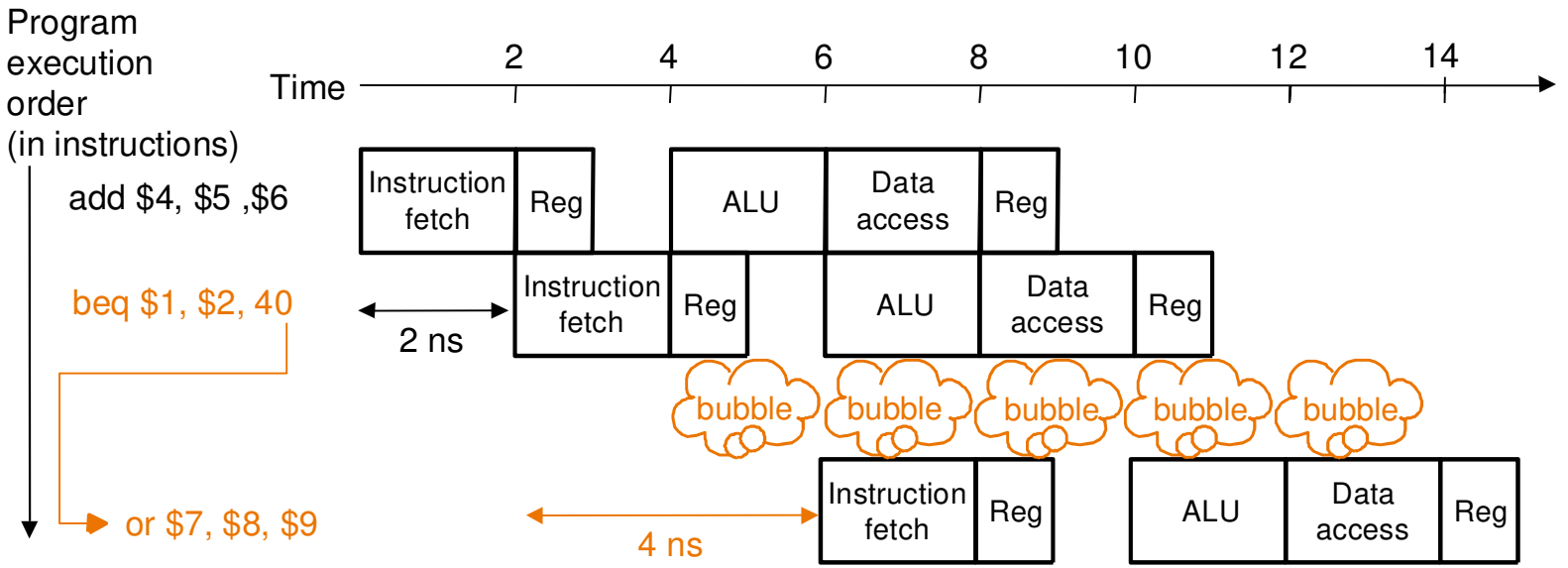


- **Dynamic scheme:** adjusting the next prediction depending on the success of recent guesses.

Predicting Condition for Branch Instruction

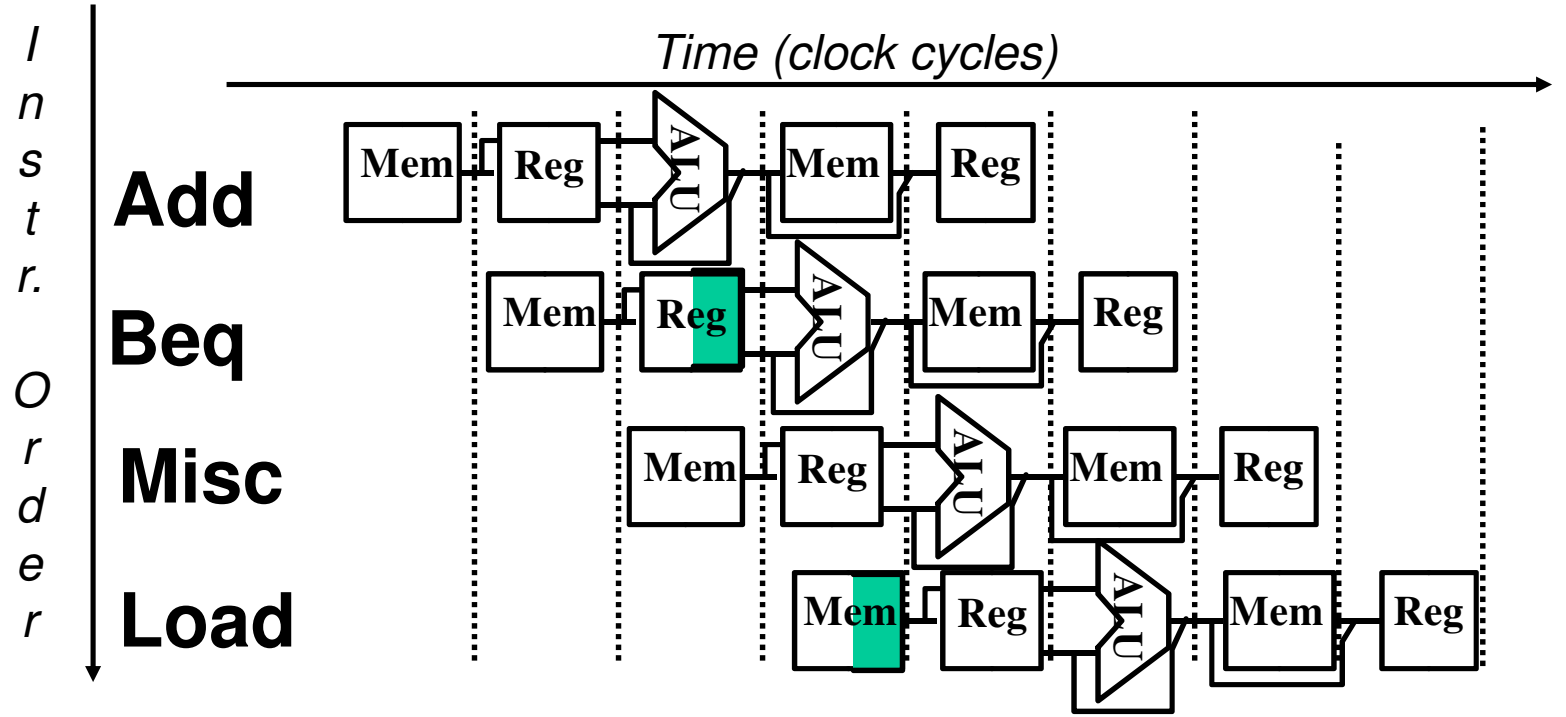






Delay Solution for Control Hazard

- Redefine branch behavior (takes place after next instruction) "delayed branch"



Delayed Branch Solution

